

五个提案

覆盖应用从创建、准入、发布、运行到故障响应的完整生命周期



01

IntentOps · 会思考的 Operator

把 LLM 装进 Reconcile 循环

创建阶段

风险：高

产品定位

一个自定义资源，spec 里写的不是字段，而是一句自然语言：`intent: "我要一个高可用的留言板，标题叫毕业纪念，前端至少两个副本"`。控制器在 Reconcile 时调用 LLM，把这句话解析成参数（标题、副本数、要不要 PDB），再协调出对应的 K8s 资源。声明式的下一站：意图式（intent-based）。

前沿锚点

"intent-driven infrastructure" 是 Operator 模式与 LLM 结合的最前沿探索，学术和工业界都刚起步——这是五个提案里概念上最超前的。

安全约束（必须写死）

LLM 只允许填一个固定模板的参数（从有限选项里选），绝不允许生成任意 YAML——这既是安全设计，也让工程量骤降。基座直接用 L5 lab 的 GuestbookApp operator，改的是"参数从哪来"这一段。

演示

`kubectl apply` 一句人话，整套应用长出来；改一句话，系统跟着变。

风险与纪律：高（要动 Go 代码 + kubebuilder）。L5 lab 原版是保底，LLM 解析是冲刺，卡住即冻结。

产品定位

与其等应用坏了再诊断，不如在坏配置进入集群之前就拦下它。实现一个准入 Webhook：任何人 `kubectll apply` 任何对象，请求在写入 etcd 之前先被服务拦截，AI 审查这份 `YAML`——没配探针？没设资源限制？容器要求特权模式？——然后放行、警告、或直接拒绝，并附一段人话解释"为什么不行、该怎么改"。相当于给集群装了一个 AI 安检门。

前沿锚点

双重前沿——"策略即代码"（OPA/Kyverno，均为 `CNCF` 毕业项目）是成熟赛道，而"用 `LLM` 做准入审查"是这个赛道正在发生的新演化。它用的是 `admission webhook` 这个扩展点（区别于 `CRD`、`API`），补齐了对 `K8s` 扩展机制的覆盖。

自研核心与工程要点

一个 `HTTPS` 服务（收 `API server` 转来的审查请求）+ 一份 `ValidatingWebhookConfiguration` + `LLM` 审查逻辑。两个必须写死的安全设计：`failurePolicy: Ignore`（服务挂了不能连累集群正常工作）、`Webhook` 只作用于打了特定标签的命名空间（绝不拦系统组件）。已知的坑：`Webhook` 强制要求 `TLS` 证书，有成熟的自签名脚本方案，但这是第一天必须验证的技术尖峰。

演示与海报

现场 `apply` 一份"裸奔"的 `Deployment`（无探针无限制）→ 被当场拒绝，终端里打印出 AI 写的整改意见；修好再 `apply` → 放行。观感是"集群会顶嘴了"。

风险与降级：中高（卡在 `TLS` 那一环）。降级路径清晰：退成一个"apply 前的预检工具"（网页里贴 `YAML`，AI 出审查报告）——丢掉"强制拦截"但保住全部审查能力。

产品定位

新版本上线，怎么保证它不把用户全炸了？新版本（v2）先只接 10% 的真实流量，一个"AI 裁判"持续对比新旧版本的错误率和延迟——v2 表现好，自动把流量比例逐级推到 50%、100%（晋升）；v2 出问题，自动把流量瞬间切回旧版（回滚），并生成一份人话事故报告。发布，从"祈祷式全量上线"变成"有裁判守着的渐进过程"。

前沿锚点

这叫"渐进式交付（progressive delivery）+ 自动金丝雀分析"，是 Argo Rollouts、Flagger（均为 CNCF 项目）正在做的事——自研一个迷你版。

核心机制

直接复用 Gateway API——HTTPRoute 的 `backendRefs` 本来就支持给多个后端配权重（weight），Envoy Gateway 环境现成支持。要写的只有"裁判循环"：给两个版本各发探测流量、统计成败、按规则改 HTTPRoute 的权重（一次 K8s API patch）。LLM 的角色设计成"规则做决策，LLM 写解说"——决策可靠性和 AI 叙事两头都占。

演示与海报

现场发布一个故意带 bug 的 v2（比如 30% 请求返回 500），大屏上看流量权重爬到 10%、裁判察觉异常、权重瞬间归零、事故报告弹出——整个过程无人干预。海报画"流量权重曲线 vs 错误率曲线"的联动图。

风险与降级：中低，工程确定性最高。降级路径体面——哪怕裁判做不完，一个"手动流量分配旋钮"的仪表盘本身已是产品。

产品定位

HPA 是反应式的——CPU 已经爆了才开始扩容，而新 Pod 启动需要时间，这段窗口里用户已经在超时了。Prophet 做的是预测式伸缩：持续记录流量的时间规律，预判"下一波高峰 90 秒后到"，提前把副本扩好等着；波峰过去，再稳妥地缩回。产品的灵魂是一场对照实验：同样一波脚本化的流量潮，反应式 HPA 那一组出现延迟尖刺，Prophet 那一组曲线平滑如常。

前沿锚点

AI 驱动的资源管理（predictive autoscaling）是各大云厂商和 KEDA 社区的活跃方向——"让容量决策从被动响应变成主动预判"。

自研核心

三件套——指标采集（装个 metrics-server，一条 Helm 命令）、预测器（第一版用简单的周期检测/移动平均就足够打赢反应式 HPA，LLM 负责把预测依据写成人话解说；有余力再让 LLM 参与预测本身）、执行器（带最小/最大副本护栏的 K8s API patch）。工程分层："简单算法保底，AI 增强叙事"。

演示与海报

海报的主视觉是双组对照图："负载曲线、副本数曲线、P95 延迟曲线"三线联动，反应式 vs 预测式并排。

风险：中。最大的不确定是 Learner Lab 上 metrics-server 的安装，以及对照实验需要可复现的流量脚本。

产品定位

一个持续值班的"AI 医生": watch 集群事件，一旦发现 Pod 进入 CrashLoopBackOff、ImagePullBackOff、OOMKilled 等状态，自动采集证据（describe、logs、events），交给 LLM 生成一份人话诊断书——"根因：探针端口写成了 8080 但容器监听 8081；建议修复：……"——实时推送到大屏。进阶版加"一键采纳修复"（带确认闸）。

前沿锚点

这正是 CNCF 沙箱项目 K8sGPT 在做的事（AIOps 的代表作）。海报上写"受 K8sGPT 启发，自研迷你版"。

为什么契合 & 演示形式

四次 lab 的排查表——那些亲手踩过的故障——直接变成产品的测试集和演示剧本。演示形式做成"红蓝对抗"：一名组员现场埋故障（改错镜像名、写错探针端口），大屏上 AI 医生 30 秒内给出诊断，观众看 AI 和故障赛跑。

风险：中低。watch + 采集 + 调 LLM，每一环都对应已学过的机制；证据采集就是把 kubectl describe/logs 程序化。课程浓度、前沿性、可行性、演示性四项全高。